CSE 585: Parallel Pipelining with Controllable Memory and WLB-LLM 4D Parallelism

Hannah Shu, Patrick Halim, Nathan Yap

Training is Expensive!

- Throughput
- Memory

Compute. Llama 3 405B is trained on up to 16K H100 GPUs, each running at 700W TDP w using Meta's Grand Teton AI server platform (Matt Bowman, 2022). Each server is equipped and two CPUs. Within a server, the eight GPUs are connected via NVLink. Training jo using MAST (Choudhury et al., 2024), Meta's global-scale training scheduler.

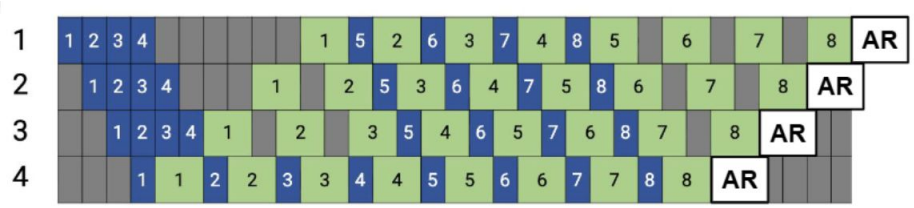
1. Pipeline Parallelism with Controllable Memory

Penghui Qi, *Sea AI Lab, National University of Singapore*; Xinyi Wan, *Sea AI Lab*; Nyamdavaa Amar, *National University of Singapore*; Min Lin, *Sea AI Lab*

2. WLB-LLM: Workload-Balanced 4D Parallelism for Large Language Model Training

Zheng Wang, *UCSD, Meta;* Anna Cai and Xinfeng Xie, *Meta*; Zaifeng Pan and Yue Guan, *UCSD*; Weiwei Chu, Jie Wang, Shikai Li, Jianyu Huang, Chris Cai, and Yuchen Hao, *Meta*; Yufei Ding, *UCSD, Meta*

Recap of Training Architecture

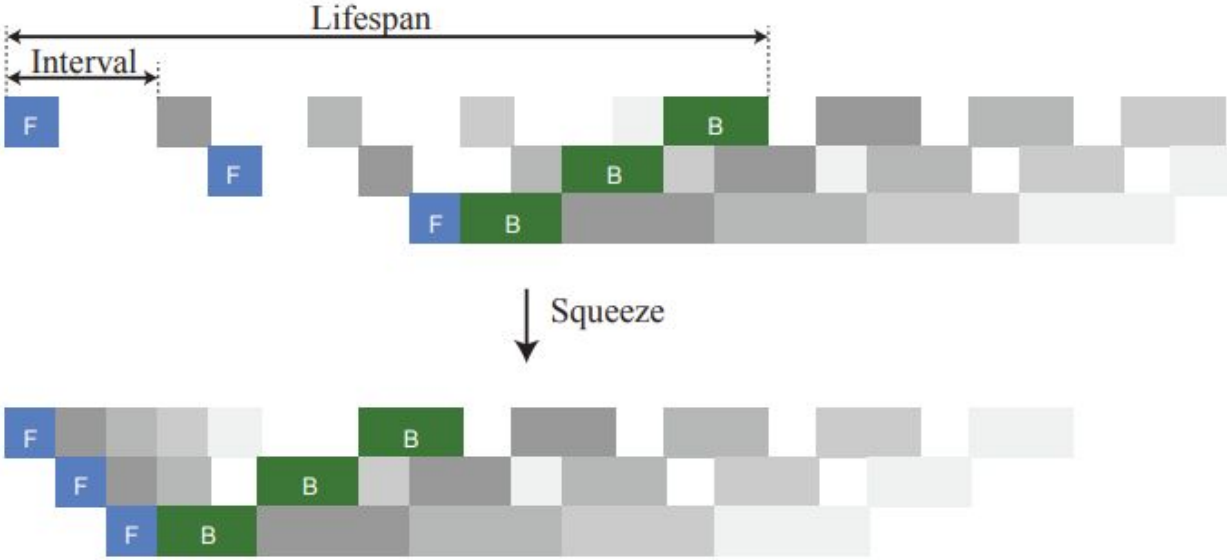


data parallelism

pipeline parallelism

https://github.com/mosharaf/cse585/blob/f25/Slides/090425-JWChung.pdf

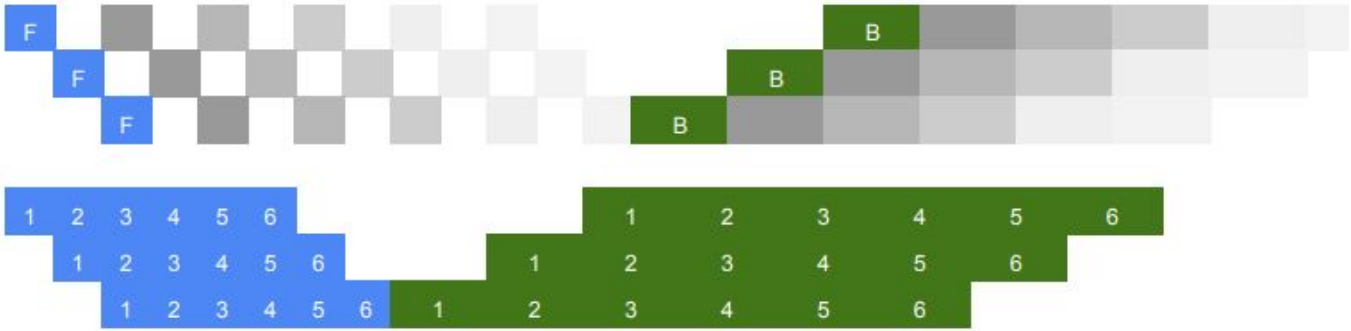
Pipeline Design Framework



- Building block
- Repeating
- Squeezing
- Reordering

Pipeline Parallelism with Controllable Memory

Existing Pipelines: GPipe

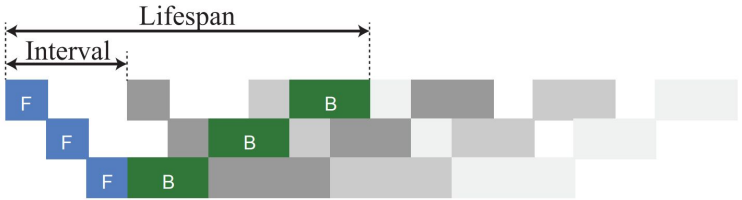


- 6 microbatches
- forward, backward
- bubble, memory

GPipe: Easy Scaling with Micro-Batch Pipeline Parallelism,
Pipeline Parallelism with Controllable Memory

Calculating Peak Memory

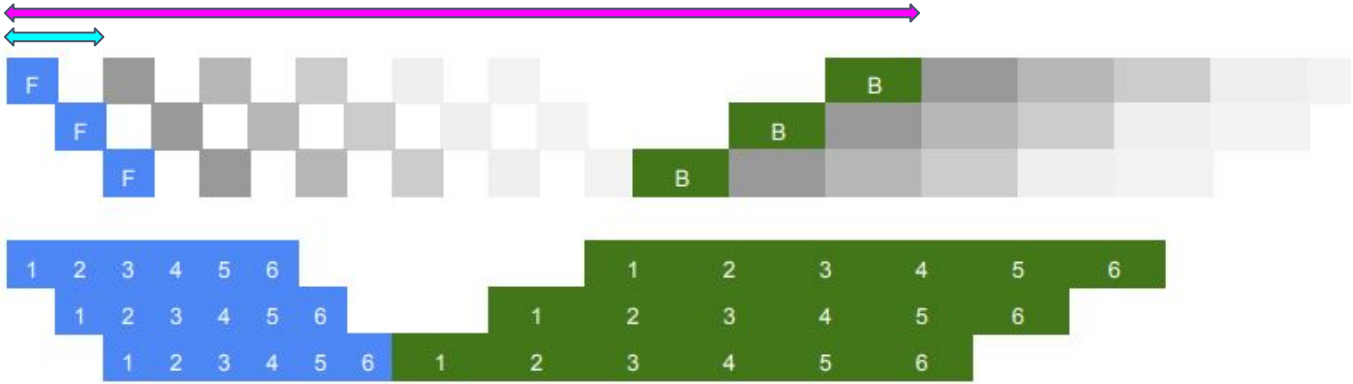
Peak memory is a function of lifespan (l), interval (T), and microbatch memory usage (m)



$$\text{peak memory} \leq \lceil \frac{l}{T} \rceil m$$

Pipeline Parallelism with Controllable Memory

Existing Pipelines: GPipe



- lifespan (l)
- interval (T)

$$\text{peak memory} \leq \lceil \frac{l}{T} \rceil m$$

[Pipeline Parallelism with Controllable Memory]

Existing Pipelines: 1F1B

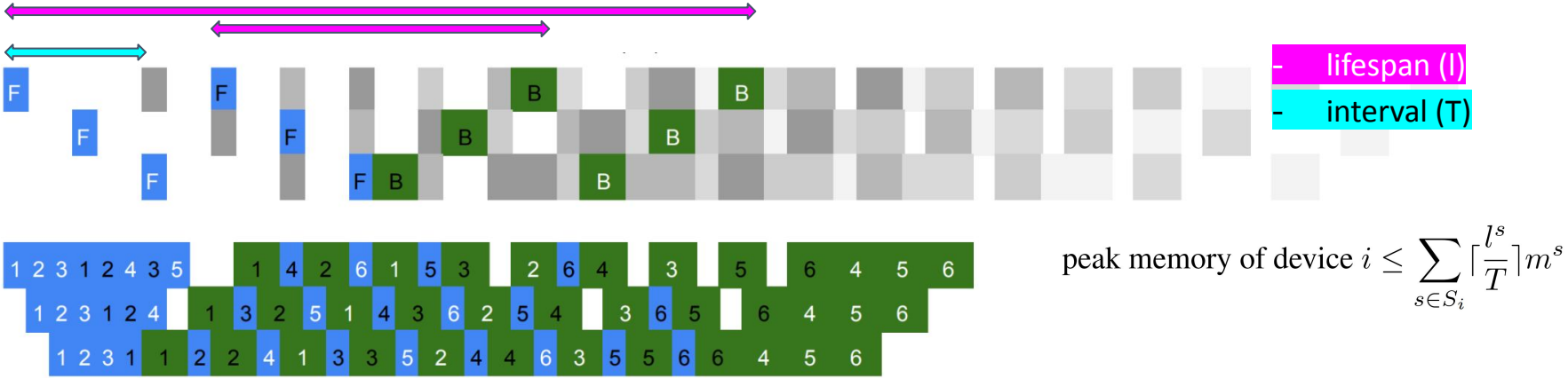
- lifespan (l)
- interval (T)

$$\text{peak memory} \leq \lceil \frac{l}{T} \rceil m$$

- decrease peak memory by starting backwards pass sooner
- same bubble size / throughput as GPipe

Pipeline Parallelism with Controllable Memory

Existing Pipelines: Interleaved 1F1B

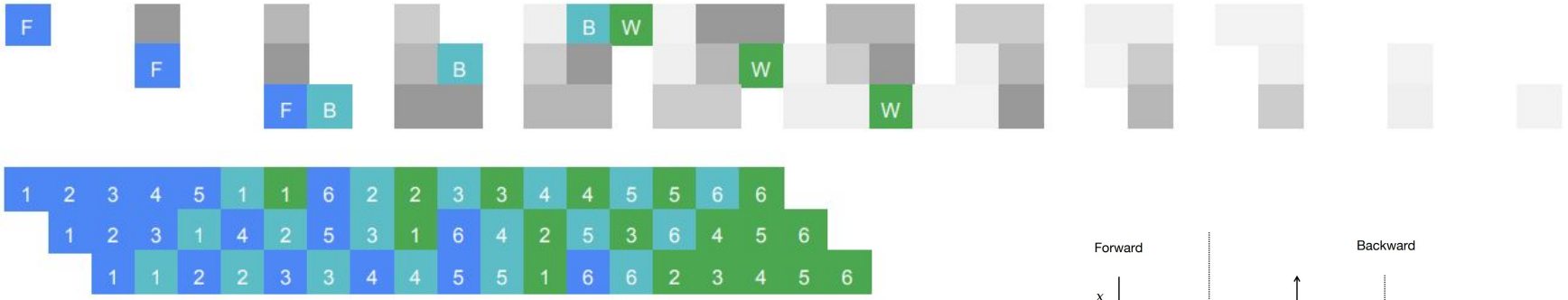


- split stages further within each device
- reduces the bubble size

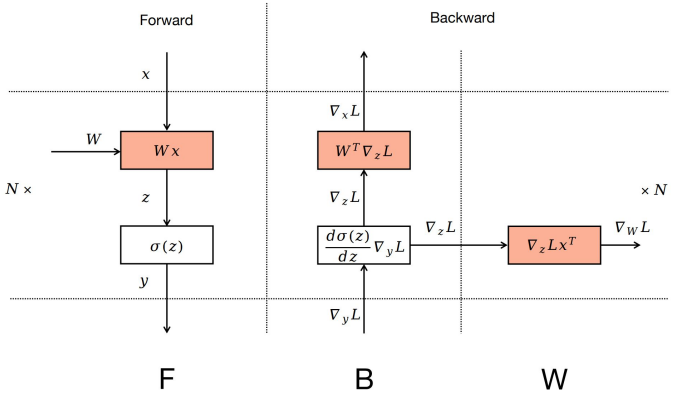
more information: [Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM](#) sec 2.2.2

[Pipeline Parallelism with Controllable Memory](#)

Existing Pipelines: ZB-H2

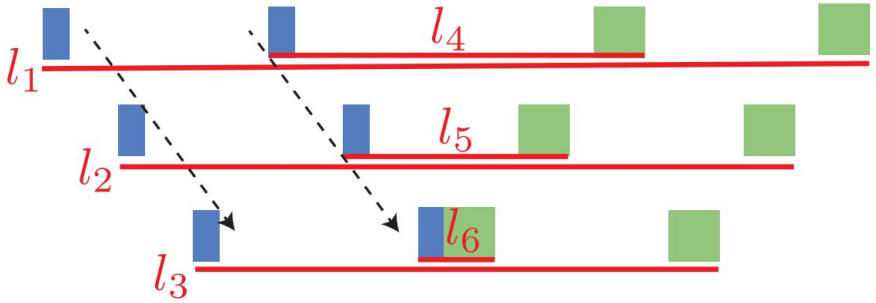


- separate calculation for B and W
- B = backward for activations
- W = backward for weights



Zero Bubble (Almost) Pipeline Parallelism,
Pipeline Parallelism with Controllable Memory

Problem: Imbalance peak memory between stages



Parallel

Interleaved 1F1B Schedule

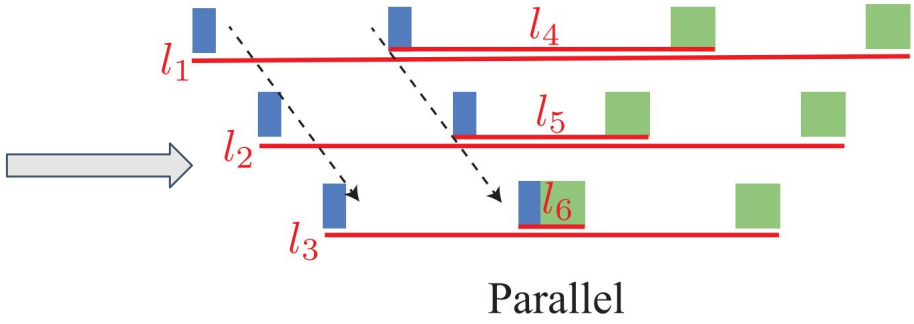
Stages have a longer cumulative lifespan in the top device. Causing a memory bottleneck

$$\text{peak memory of device } i \leq \sum_{s \in S_i} \lceil \frac{l^s}{T} \rceil m^s$$

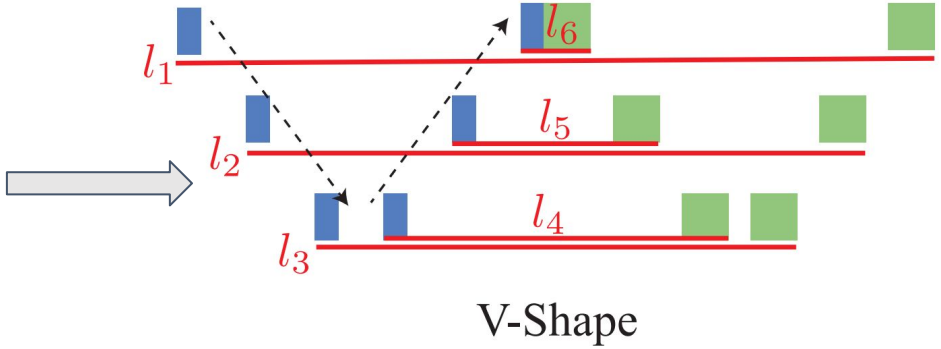
[Pipeline Parallelism with Controllable Memory]

Memory Efficient V-shaped Building Blocks

Arranging interleaved stages in **parallel** causes the top Device to have higher peak memory.

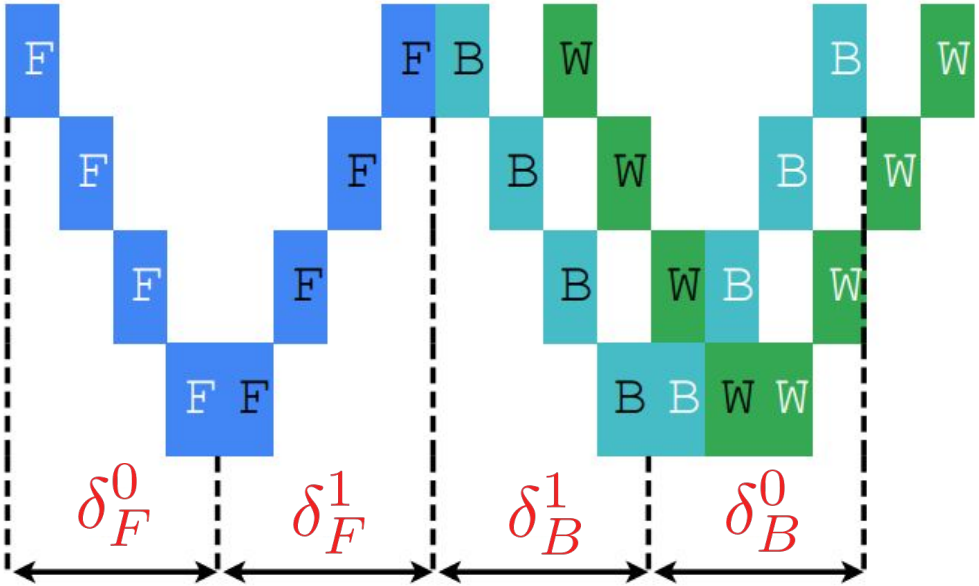


A **V-Shaped** arrangement distributes peak-memory usage more equitably between devices by splitting each stage into 2 parts



Pipeline Parallelism with Controllable Memory

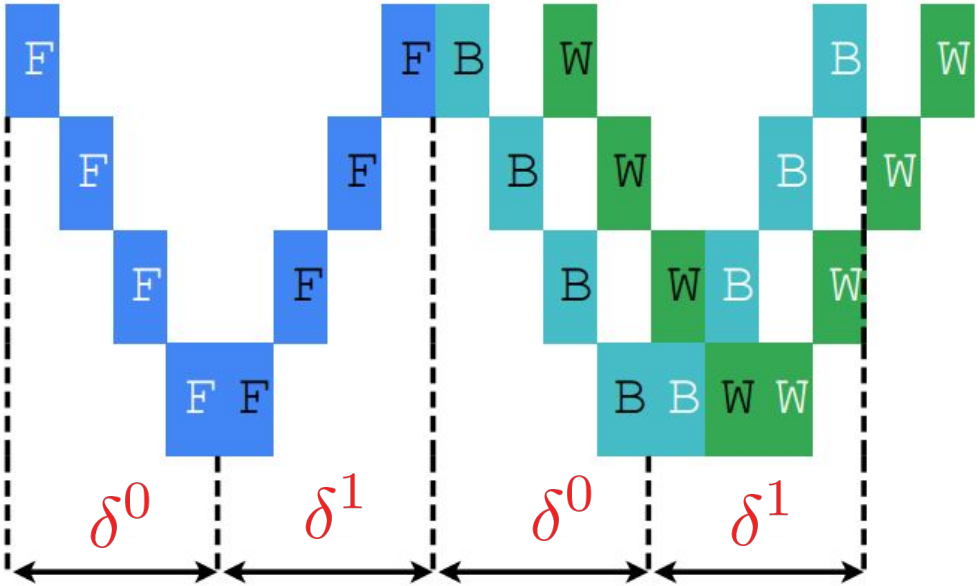
Controllable Balanced Memory



The offset between blocks can balance peak memory and throughput

[Pipeline Parallelism with Controllable Memory]

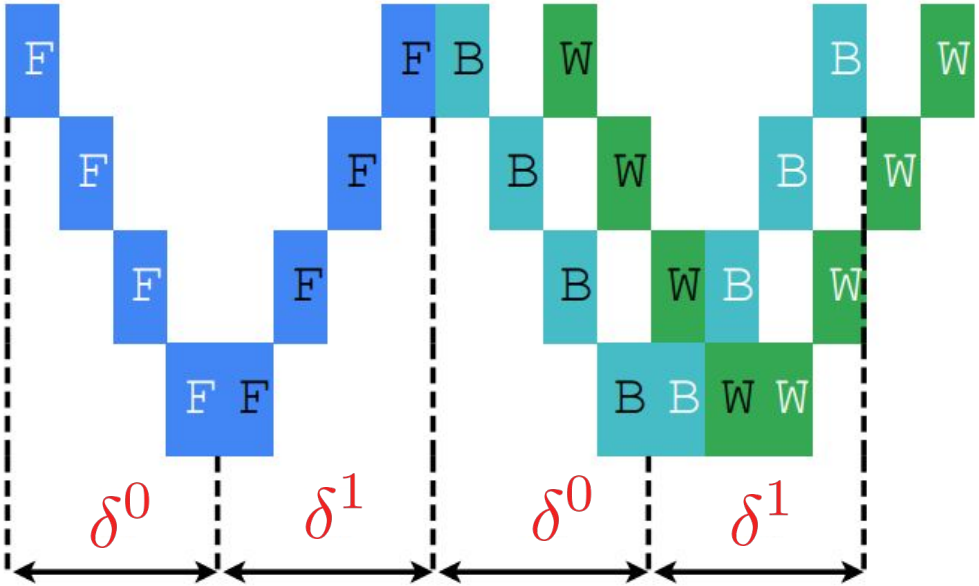
Controllable Balanced Memory



Match the offset between the corresponding F/B passes

[Pipeline Parallelism with Controllable Memory]

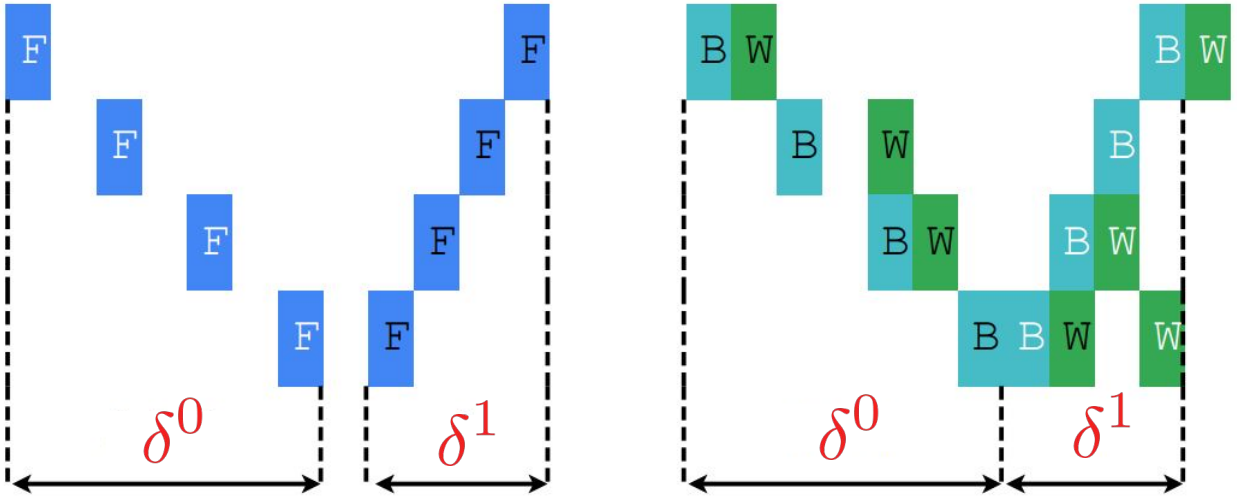
Controllable Balanced Memory



V-min building block

[Pipeline Parallelism with Controllable Memory]

Controllable Balanced Memory

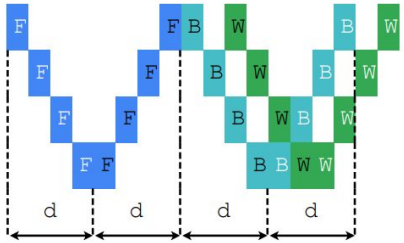


The **V-half** building block doubles the first offset

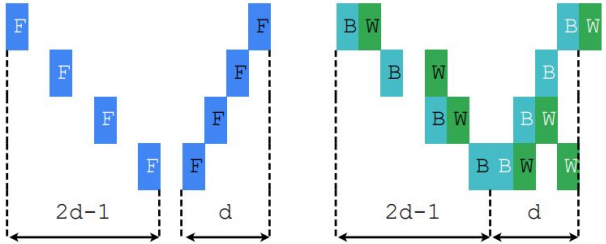
[Pipeline Parallelism with Controllable Memory]

Controllable Balanced Memory

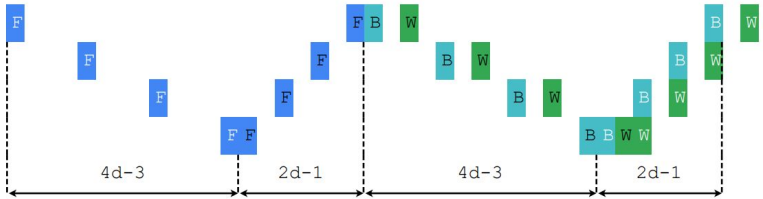
By varying offset, three different V-shaped building blocks were created.



(c) V-Min



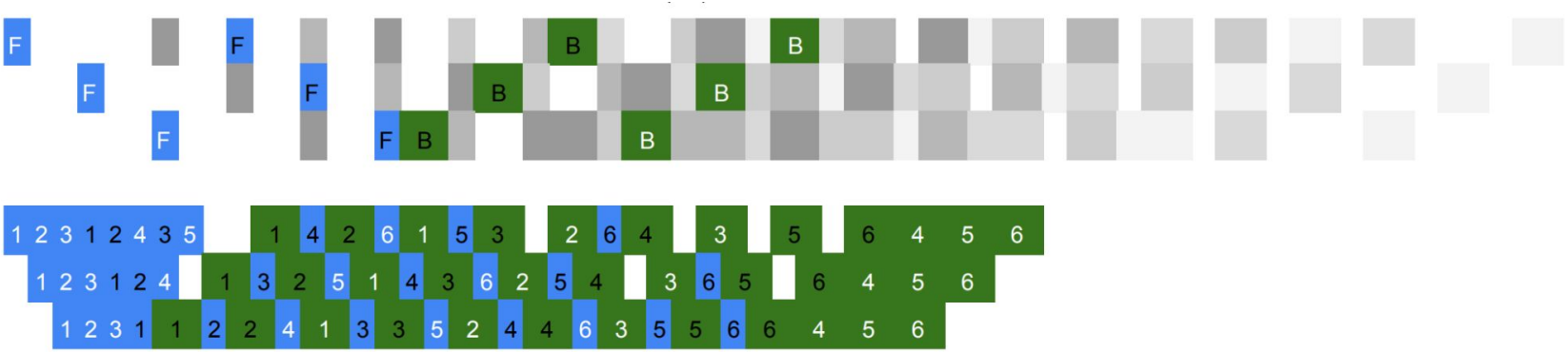
(b) V-Half



(d) V-ZB

[Pipeline Parallelism with Controllable Memory]

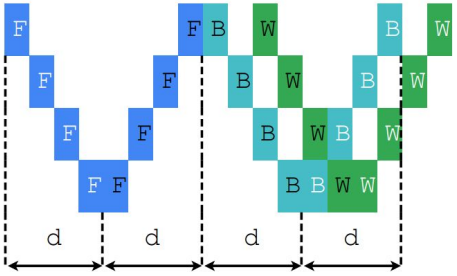
Controllable Balanced Memory



Interleaved 1F1B schedule

[Pipeline Parallelism with Controllable Memory]

Controllable Balanced Memory



(c) V-Min

$$\delta^0 = 1$$

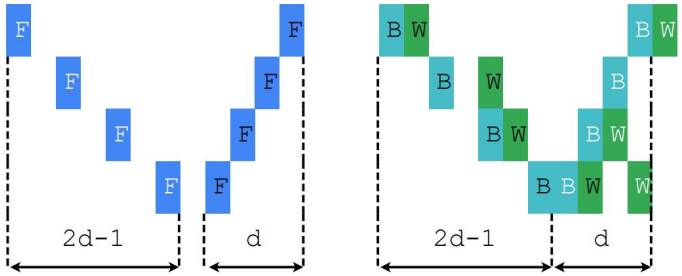
$$\delta^1 = 1$$

Tradeoff lower peak memory for more bubbles

(b) V-Min

Pipeline Parallelism with Controllable Memory

Controllable Balanced Memory

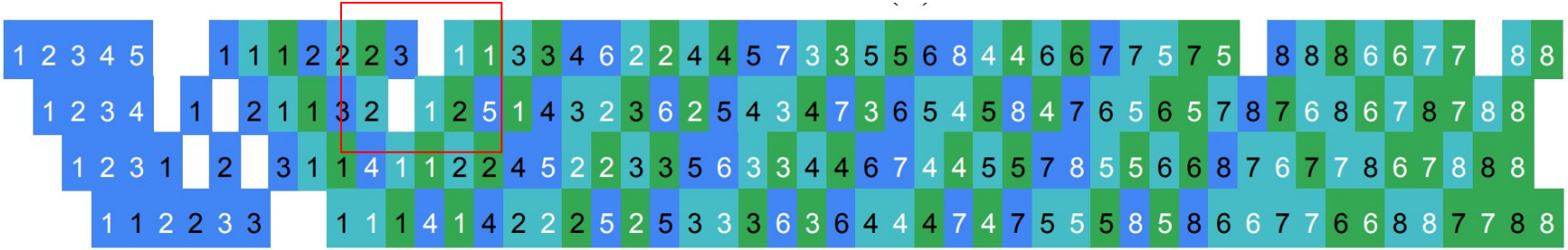


(b) V-Half

$$\delta^0 = 2$$

$$\delta^1 = 1$$

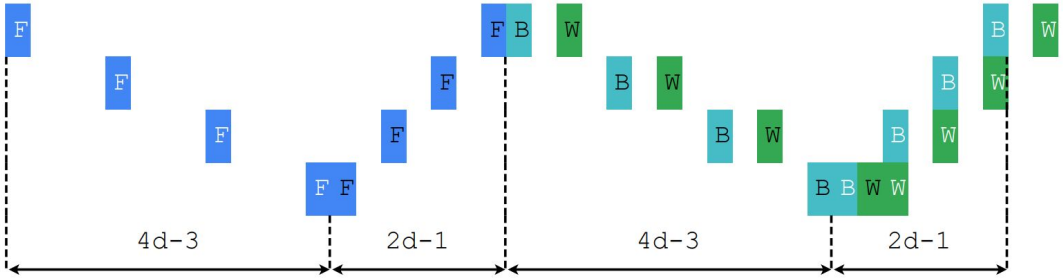
Slightly higher peak memory for fewer bubbles



Pipeline Parallelism with Controllable Memory

(c) V-Half

Controllable Balanced Memory



$$\delta^0 = 4$$

$$\delta^1 = 2$$

(d) V-ZB

High throughput (Zero Bubbles) with comparable peak memory to 1F1B

Pipeline Parallelism with Controllable Memory

(d) V-ZB

Controllable Balanced Memory

$$\text{peak memory of device } i \leq \frac{2d(\delta^0 + \delta^1) + O(1)}{6} m \approx \frac{\delta^0 + \delta^1}{6} M$$

Building Block	δ^0	δ^1	Peak Memory	Bubbles
1F1B	2	4	M	$\approx 6d$
V-Min	1	1	$\approx M/3$	$\approx 4d$
V-Half	2	1	$\approx M/2$	$\approx 3d$
V-ZB	4	2	M	0

Pipeline Parallelism with Controllable Memory

Experimental Configuration

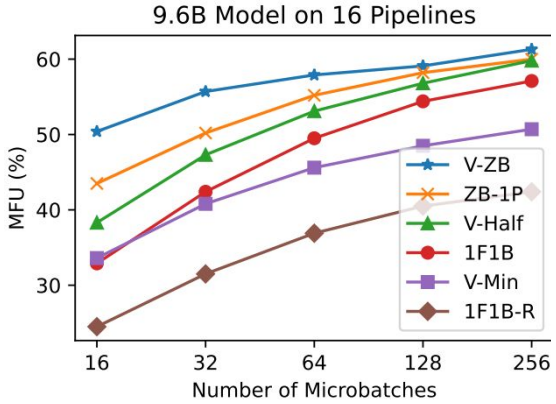
Ran experiments on 40 NVIDIA A100 SXM 80G GPUs based on the Megatron-LM project.

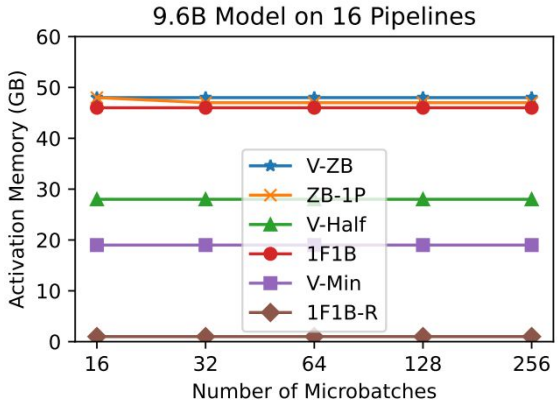
Model	Layers	Attention Heads	Hidden Size	GPUs
9.6B	30	40	5120	16
21B	46	48	6144	24
38.5B	62	64	7168	32
98.5B	78	80	10240	40

Pipeline Parallelism with Controllable Memory

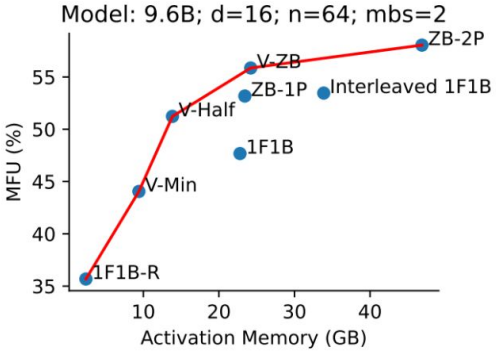
Experimental Results

V-shaped schedules had higher efficiency and Lower Activation Memory





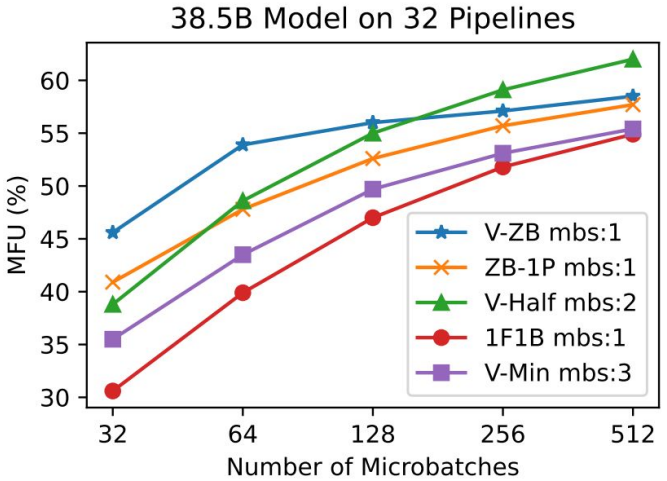
They also show the best tradeoff between efficiency and memory



[Pipeline Parallelism with Controllable Memory]

Experimental Results

Lower peak memory under similar memory constraints allows for **increased microbatch size** and higher arithmetic intensity



[Pipeline Parallelism with Controllable Memory]

Discussion

Strengths

- increased throughput and decreased activation memory for v-half
- created a framework for pipeline design that allows for controllable memory optimization

Weaknesses

- v-min has a repeating bubble that scales with the number of microbatches, limitation of current framework

Future Work

- remove v-min bubbles with different types of offsets

1. Pipeline Parallelism with Controllable Memory

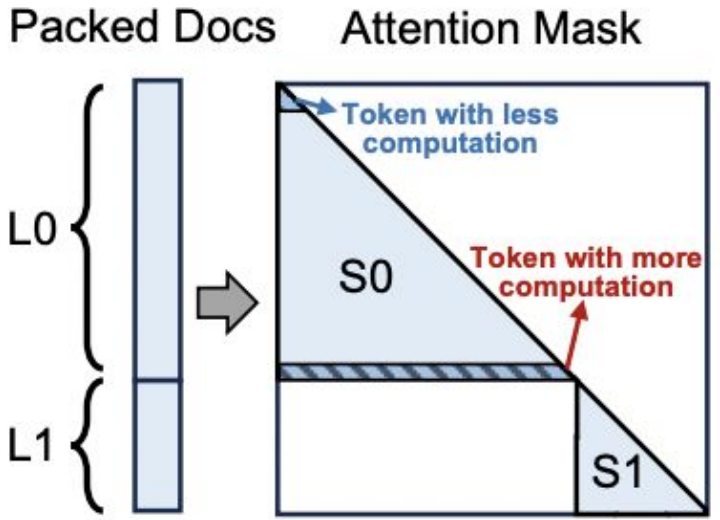
Penghui Qi, Xinyi Wan, Nyamdavaa Amar, Min Lin

2. WLB-LLM: Workload-Balanced 4D Parallelism for Large Language Model Training

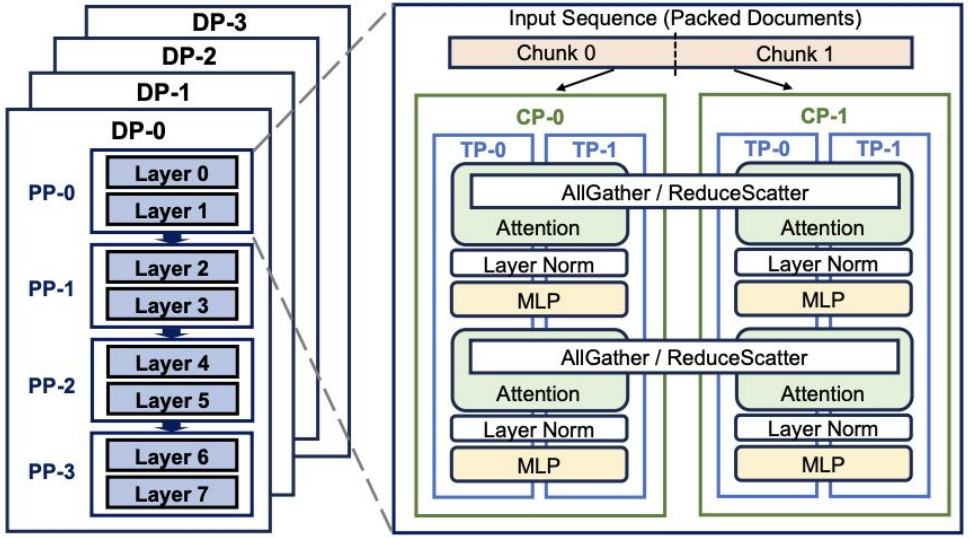
Zheng Wang, Anna Cai, Xinfeng Xie, Zaifeng Pan, Yue Guan, Weiwei Chu, Jie Wang, Shikai Li, Jianyu Huang, Chris Cai, Yuchen Hao, Yufei Ding

Refresher: Attention Masks

- Language models are autoregressive (only attend to preceding token outputs)
- Future positions are masked out (e.x. set to -inf) to achieve this

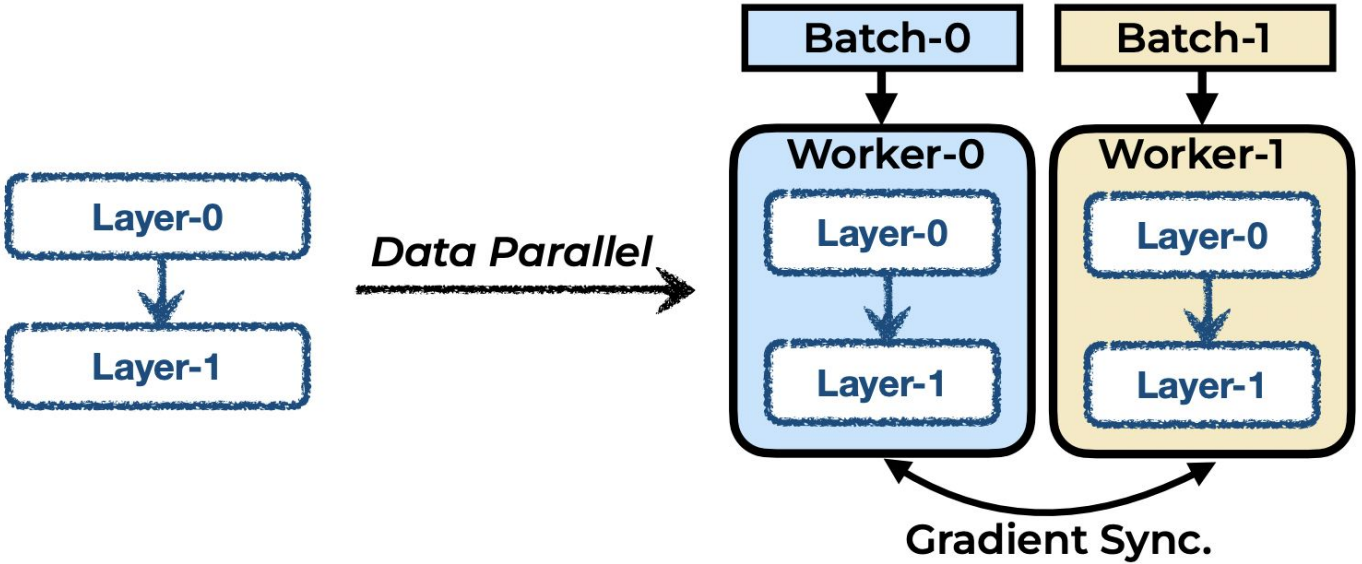


4D Parallelism

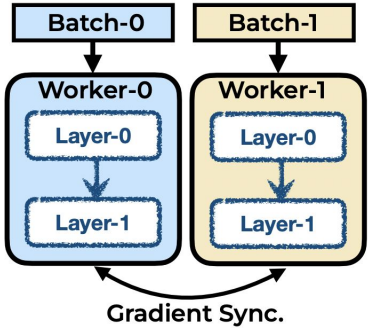


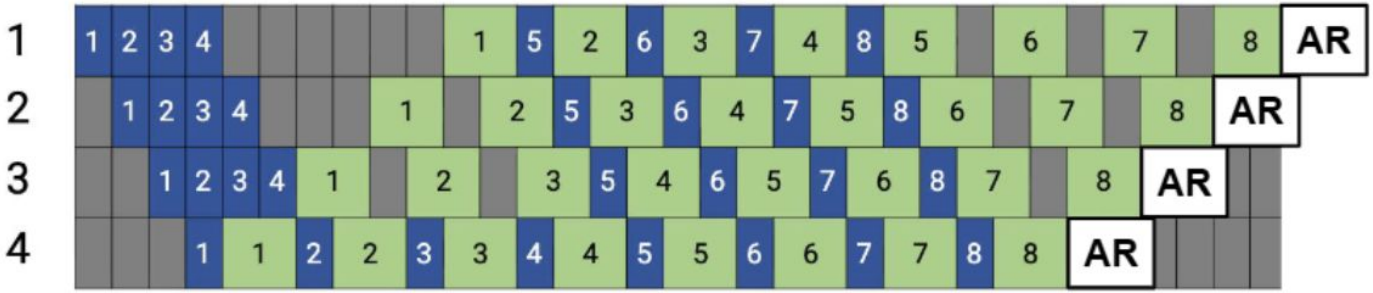
- DP - Data parallelism
- PP - Pipeline parallelism
- CP - Context parallelism
- TP - Tensor parallelism

Data Parallelism



Pipeline Parallelism

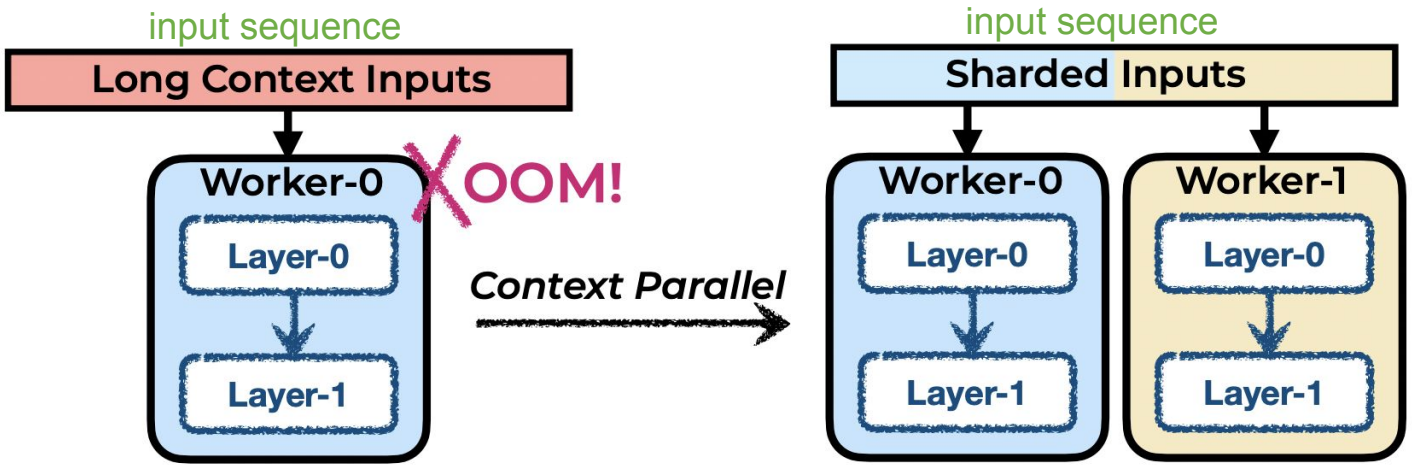




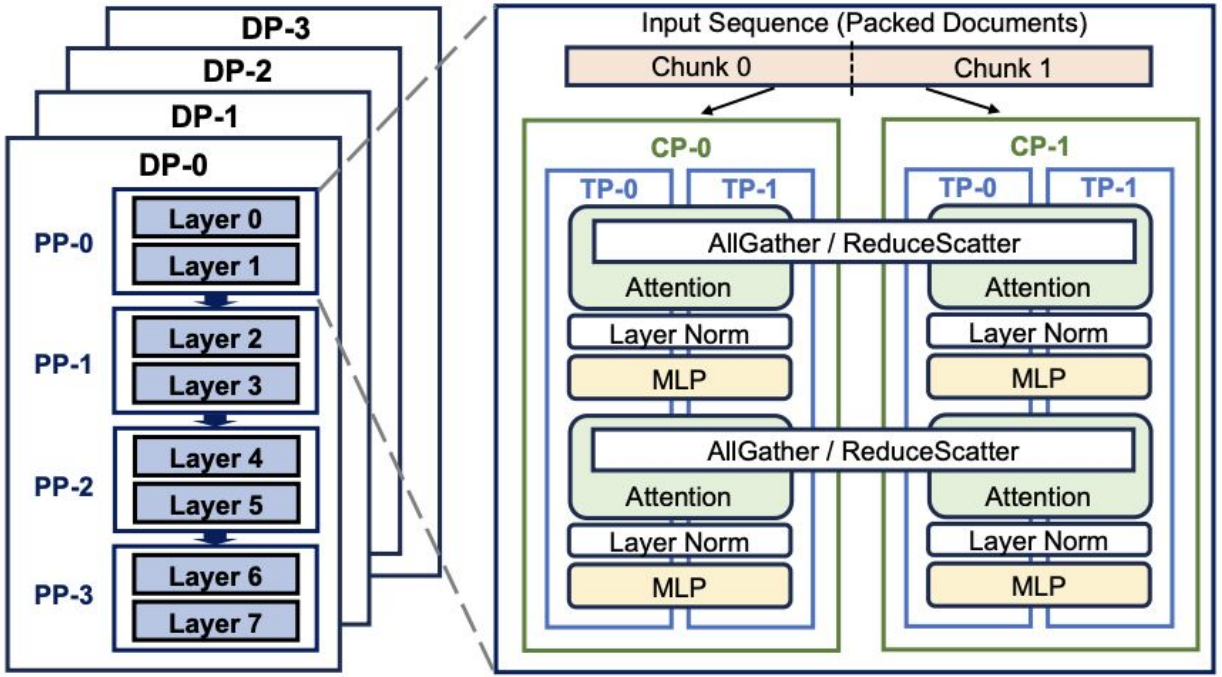
Context Parallelism

Context Parallelism:

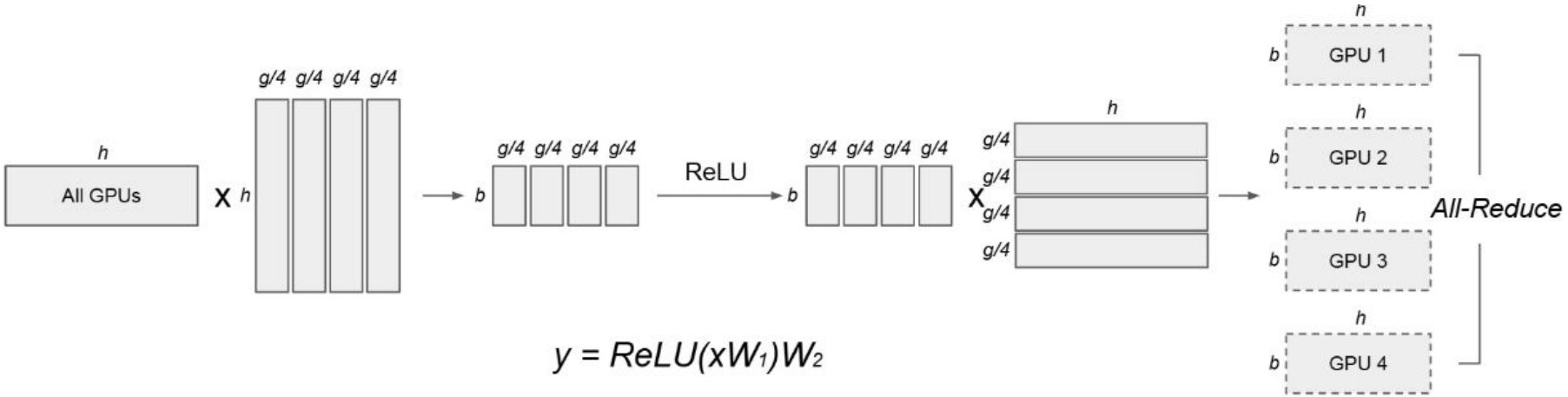
- Shard the long-context input along sequence length dimension.



Context Parallelism



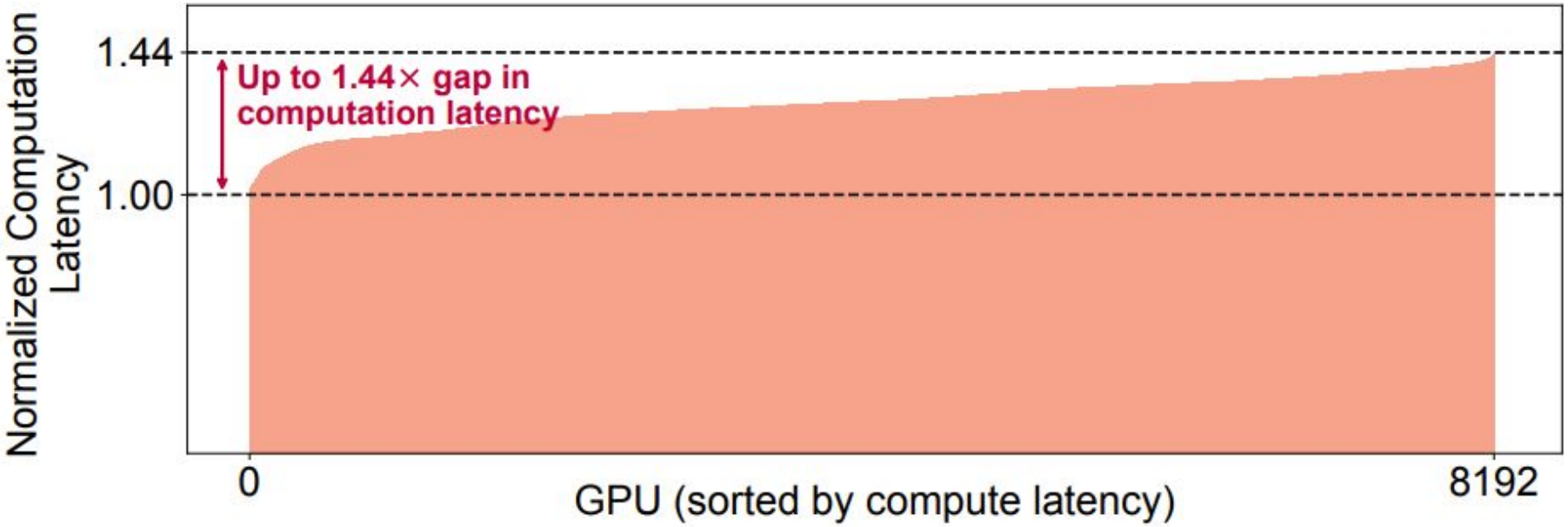
Tensor Parallelism



$$y = ReLU(xW_1)W_2$$

WLB-LLM: Workload-Balanced 4D Parallelism for Large Language Model Training

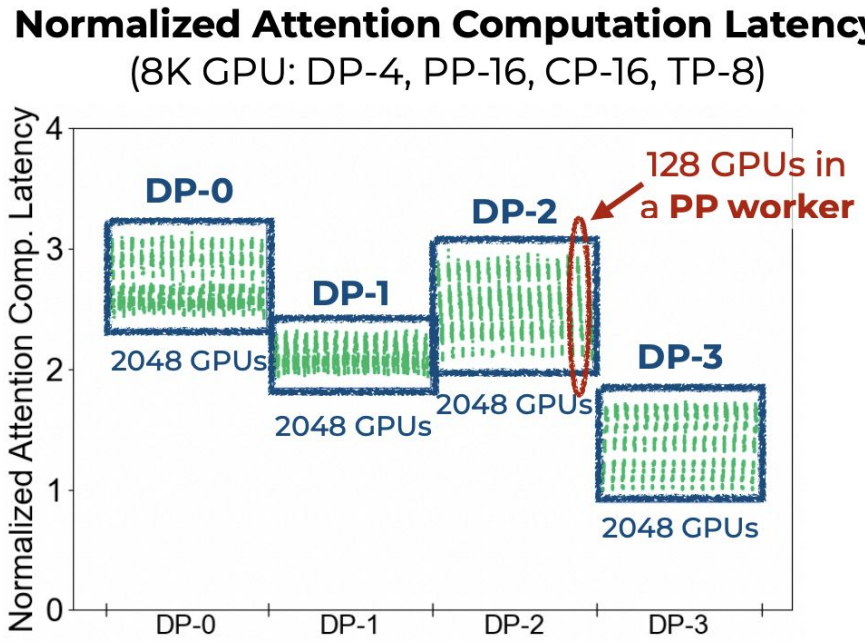
GPU Underutilization



Causes of Imbalance

Parallelism Dimension	*Imbalance? Why?*
Data Parallelism	 Input Packing
Pipeline Parallelism	
Context Parallelism	 Input Sharding
Tensor Parallelism	 No Imbalance

Reason of Imbalance: (1) Input Packing

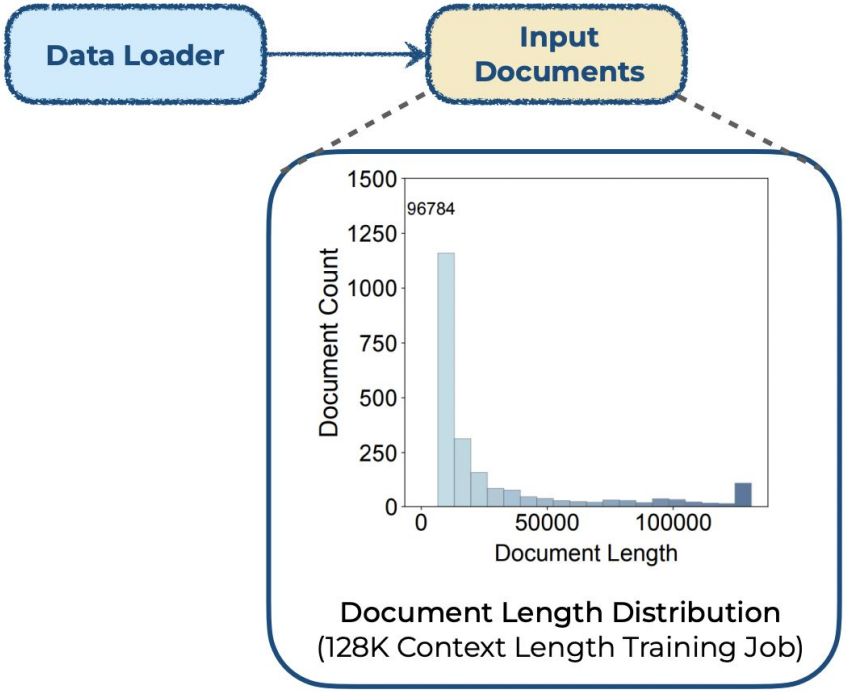


Observation:

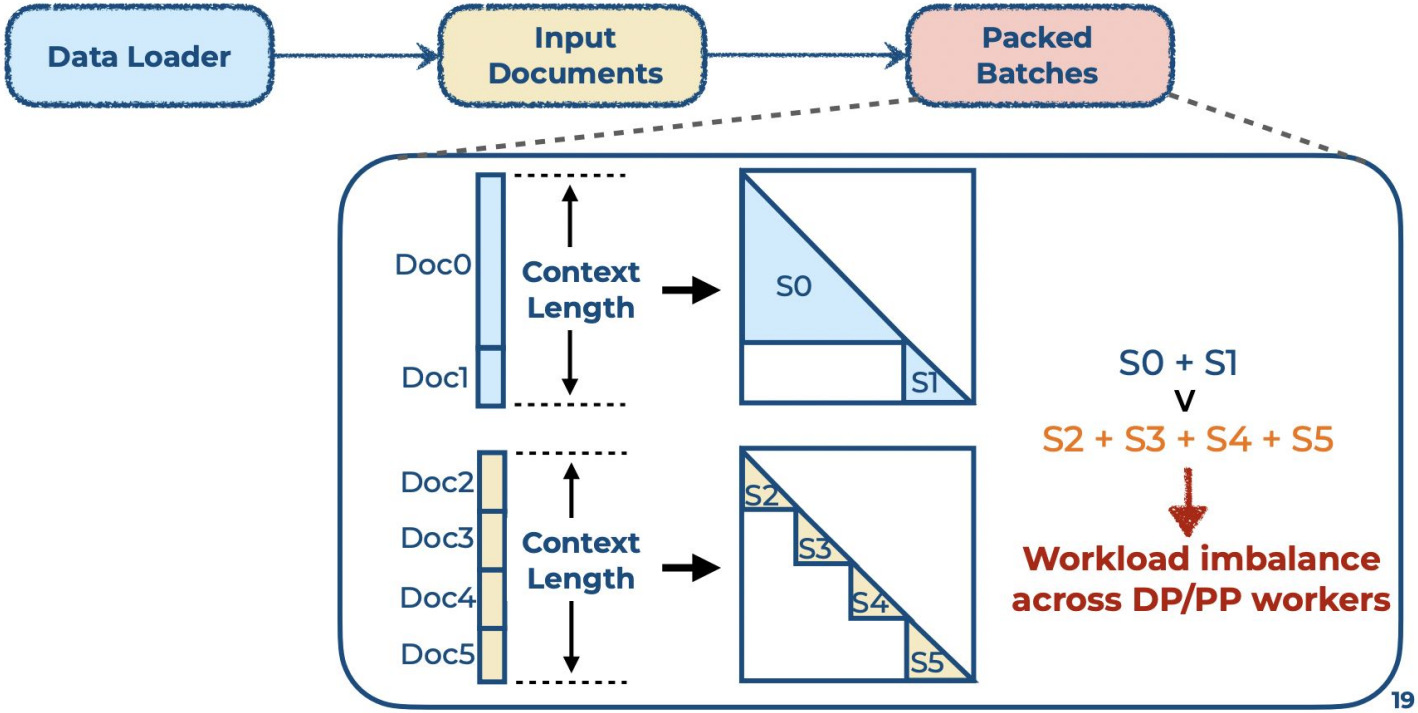
- Imbalance across **DP/PP workers**.

input packing

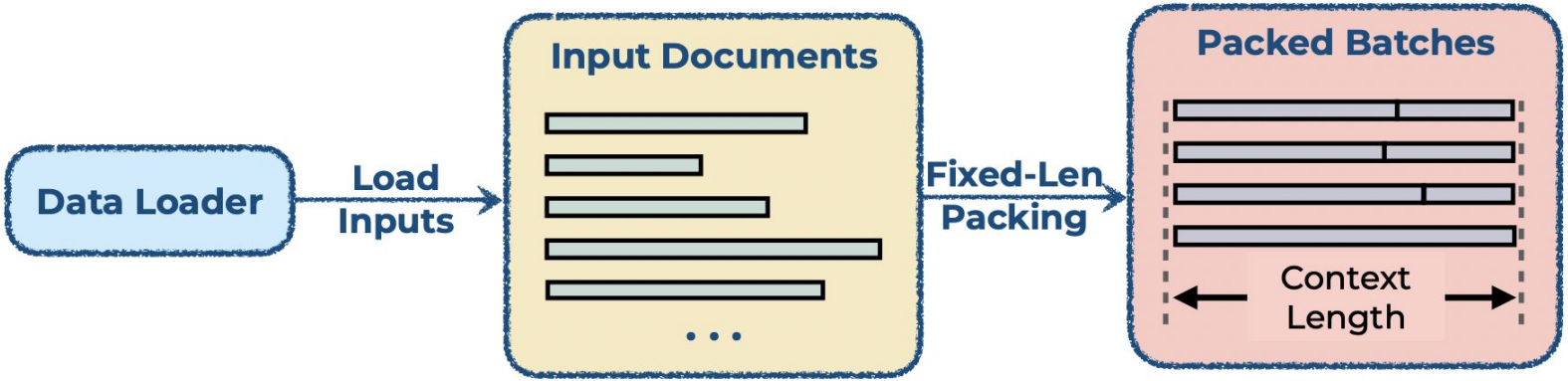
Reason of Imbalance: (1) Input Packing



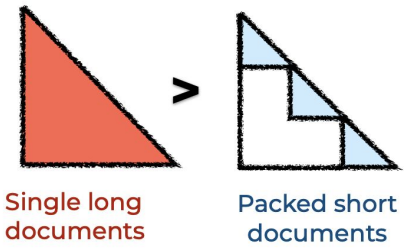
Reason of Imbalance: (1) Input Packing

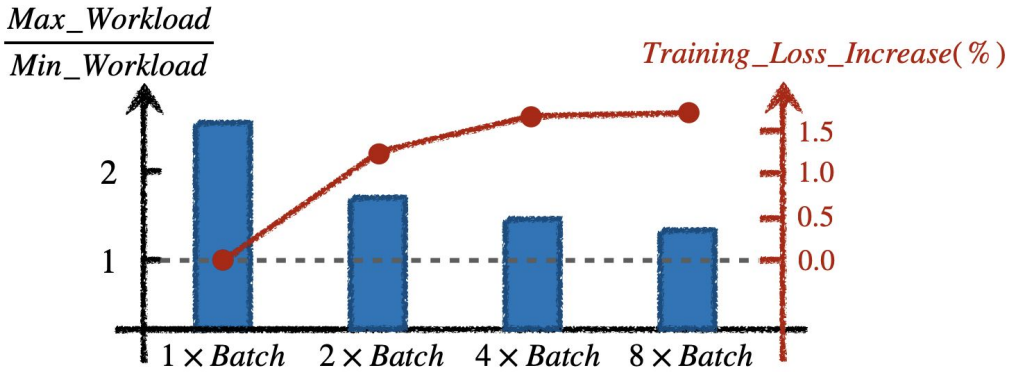


Naive Solution: Fixed-Length Packing



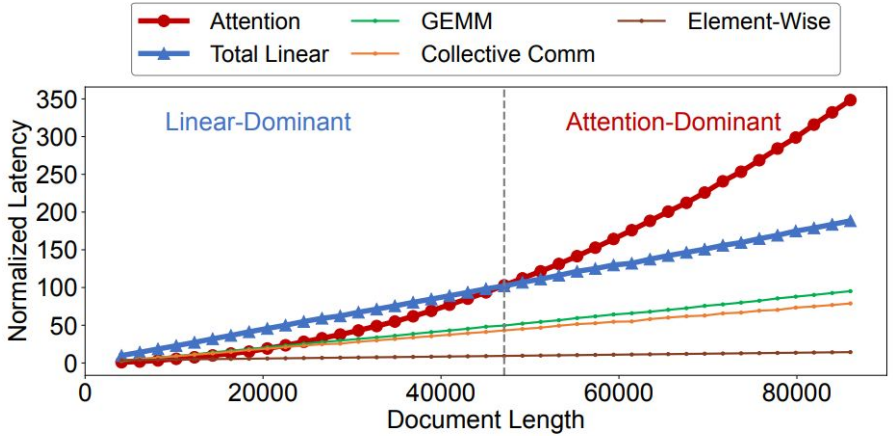
Fixed-Length Packing





Solution: Var-Len Packing + Outlier Delay

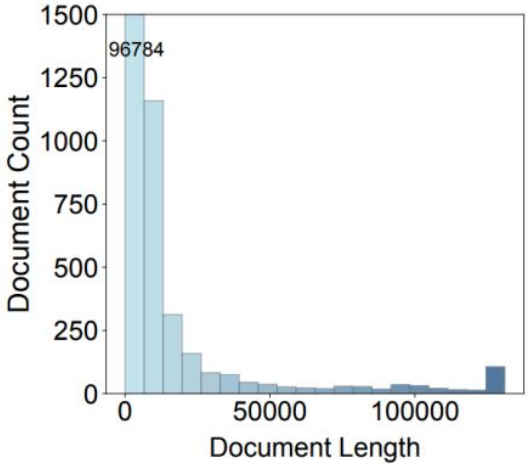
- Fixed context length → variable length
- Balance attention workload → balance total workload



Solution: Var-Len Packing + Outlier Delay

Solution: Var-Len Packing + Outlier Delay

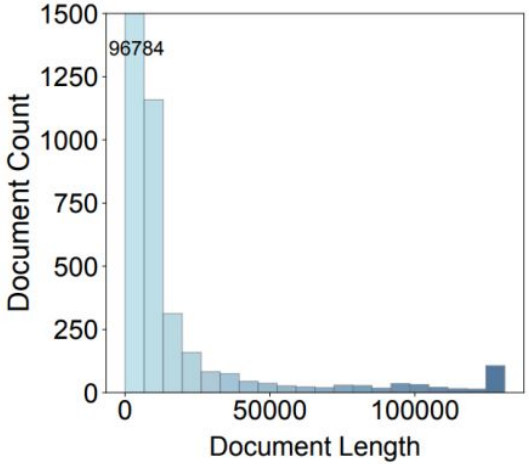
- Reorder all documents → selectively delay long documents



Long documents:

- Large impact on imbalance
- Only contributes a small ratio of training tokens

Solution: Var-Len Packing + Outlier Delay

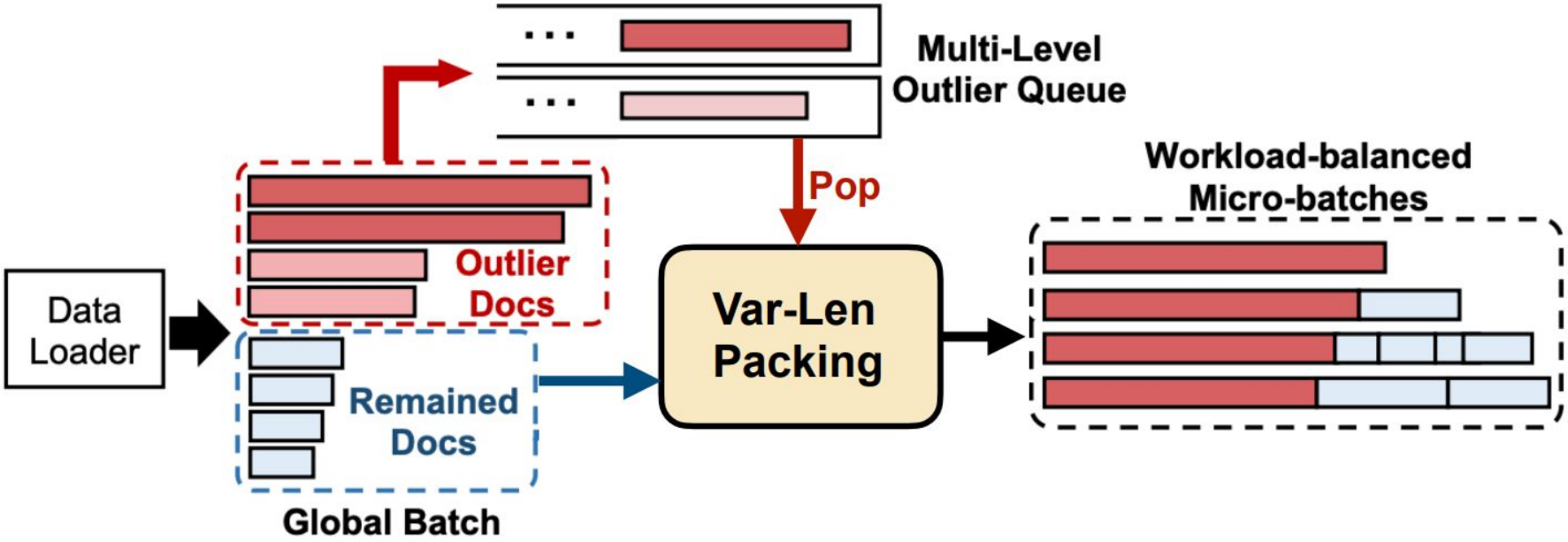


Long documents:

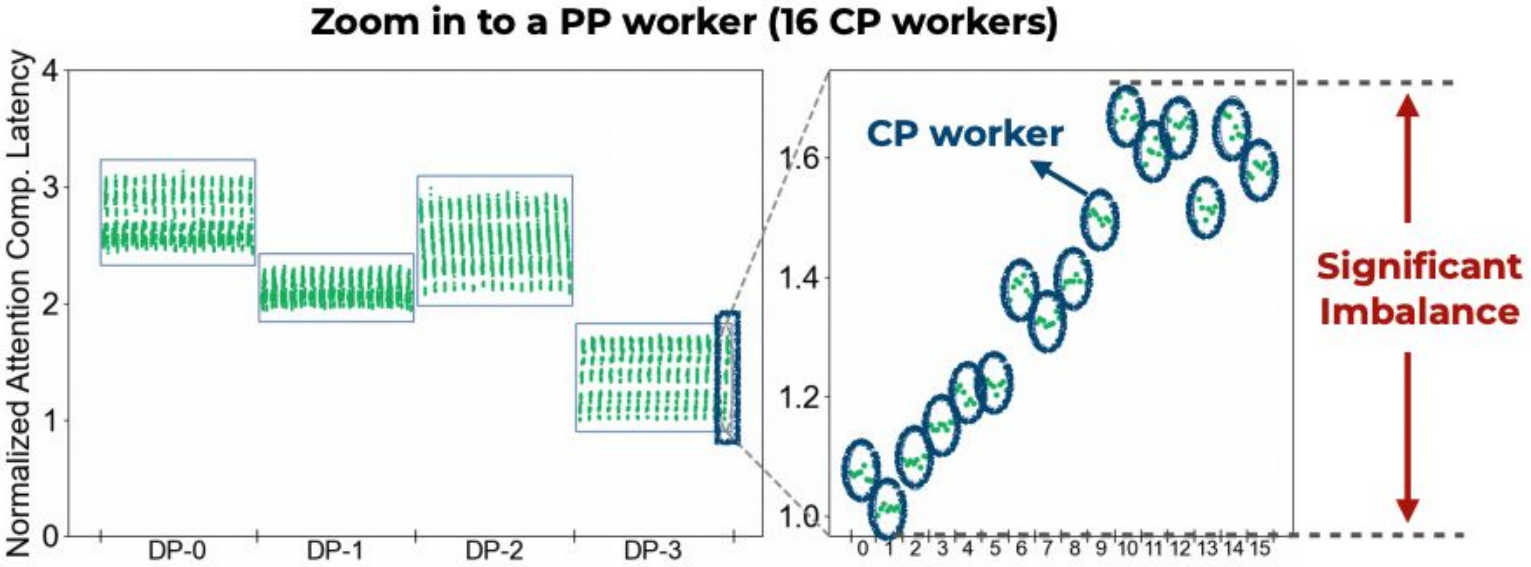
- Large impact on imbalance
- Only contributes a small ratio of training tokens

Adaptively delay the execution of outlier documents to balance their distribution across microbatches

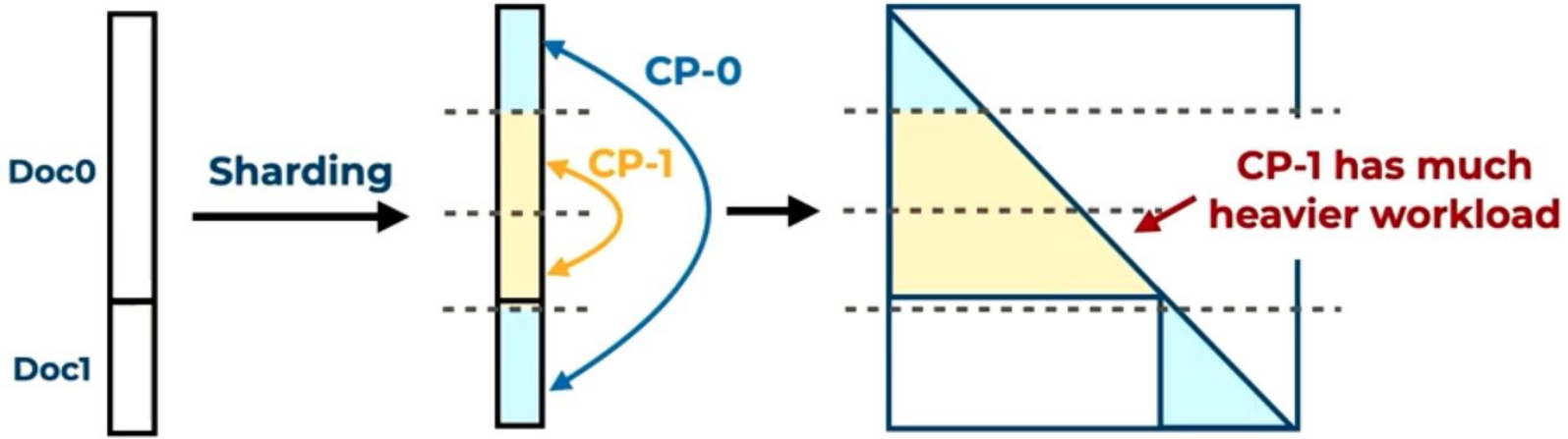
Solution: Var-Len Packing + Outlier Delay



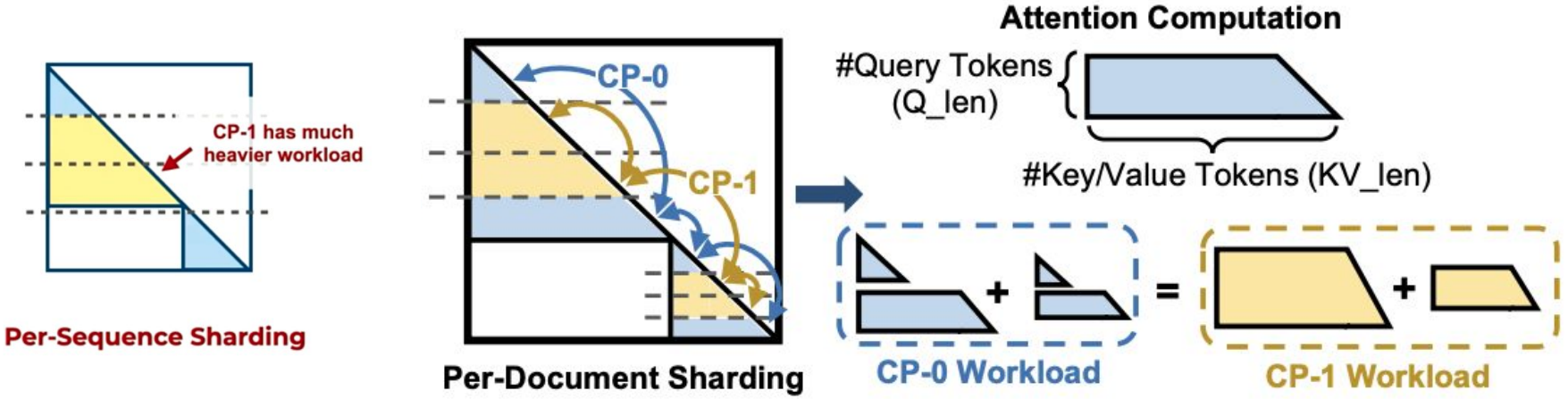
Reason of Imbalance: (2) Input Sharding



Reason of Imbalance: (2) Input Sharding

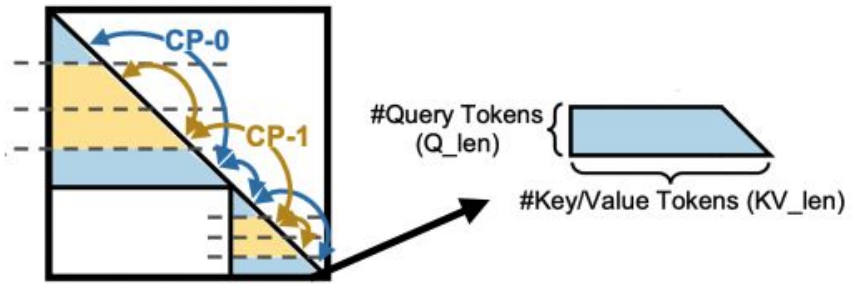


Solution: Per-Doc + Adaptive Sharding

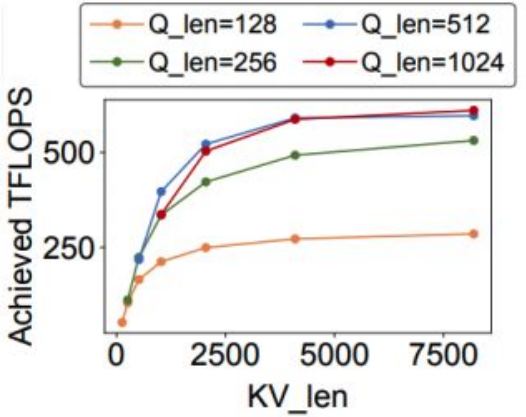


Solution: Per-Doc + Adaptive Sharding

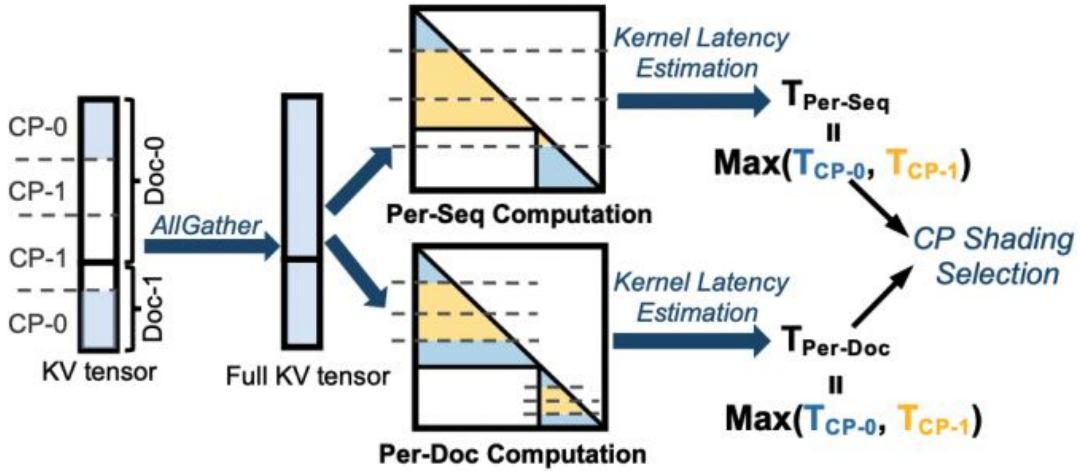
Kernel Efficiency vs. Sharding Balance:



Fine-grained sharding may leads to small document shards



Solution: Per-Doc + Adaptive Sharding



Adaptively choose the CP sharding strategy with lower latency

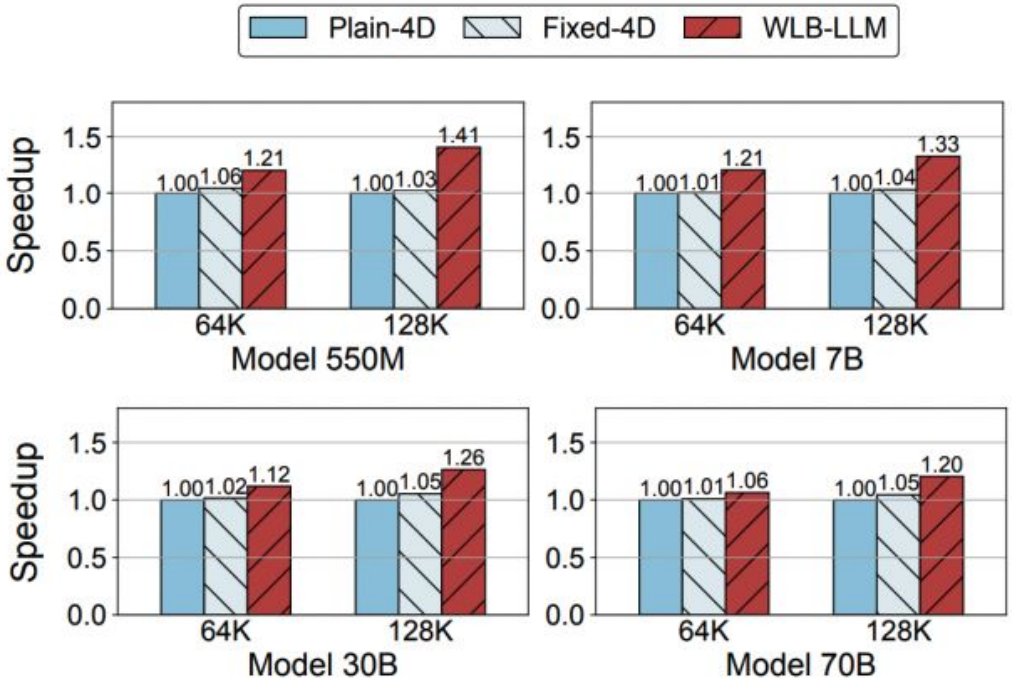
Evaluation

- **Model and 4D Parallelism Configs:**

Model Size	Context Window	#GPU	4D Parallelism Configs (TP, CP, PP, DP)
550M	*64K*	*32*	*(2, 2, 4, 2)*
	128K	*32*	*(2, 4, 4, 1)*
7B	*64K*	*32*	*(4, 2, 4, 1)*
	128K	*64*	*(8, 2, 4, 1)*
30B	*64K*	*64*	*(8, 2, 4, 1)*
	128K	*128*	*(8, 4, 4, 1)*
70B	*64K*	*256*	*(16, 4, 4, 1)*
	128K	*256*	*(16, 4, 4, 1)*

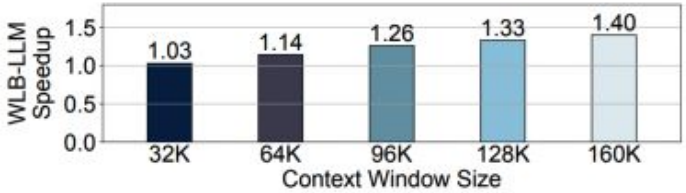
- **Baselines:**
  - **Plain-4D:** 4D parallelism implementation for Llama3 training
  - **Fixed-4D:** Plain-4D + Fixed-length Packing

Evaluation: End-to-end performance



Evaluation: Optimization Analysis

- **Larger Context Window —> Better Performance:**



- **Near Optimal + Low Overhead:**

Packing Method		Imbalance Degree	Packing Overhead (ms)
Method	**Config**		
Original Packing	/	1.44	0
Fixed-Len Greedy	#global batch=1	1.41	4
	#global batch=2	1.22	5
	#global batch=4	1.11	5
	#global batch=8	1.08	5
Fixed-Len Solver	#global batch=1	1.40	467
	#global batch=2	1.18	1488
	#global batch=4	1.09	25313
WLB-LLM	#queue=1	1.24	8
	#queue=2	1.05	20
	#queue=3	1.05	23

Summary of Optimizations

Parallelism Dimension	*Imbalance? Why?*	*Optimization*
Data Parallelism	 Input Packing	Var-Len Packing + Outlier Delay
Pipeline Parallelism		
Context Parallelism	 Input Sharding	Per-Doc and Adaptive Sharding
Tensor Parallelism	 No Imbalance	/

Discussion

Strengths

- First solution addressing workload imbalances in 4D parallelism
- Strong empirical evidence that balancing strategy leads to meaningful speed-ups

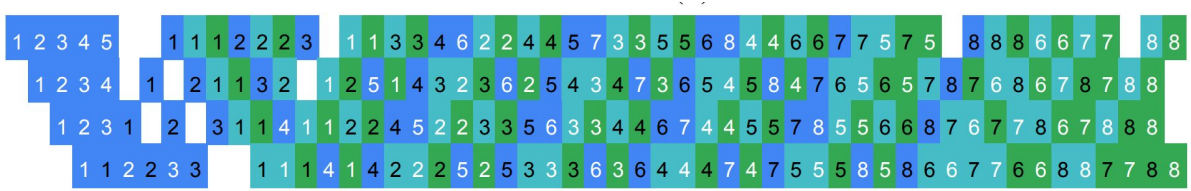
Weaknesses

- Experimental results tied to Meta's internal training pipeline
- Impact on convergence studied lightly

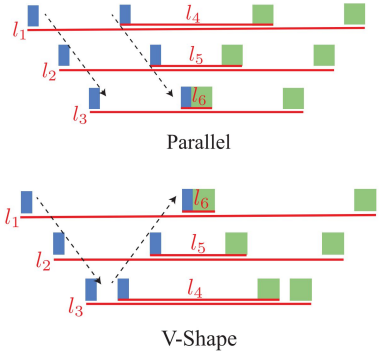
Future Work

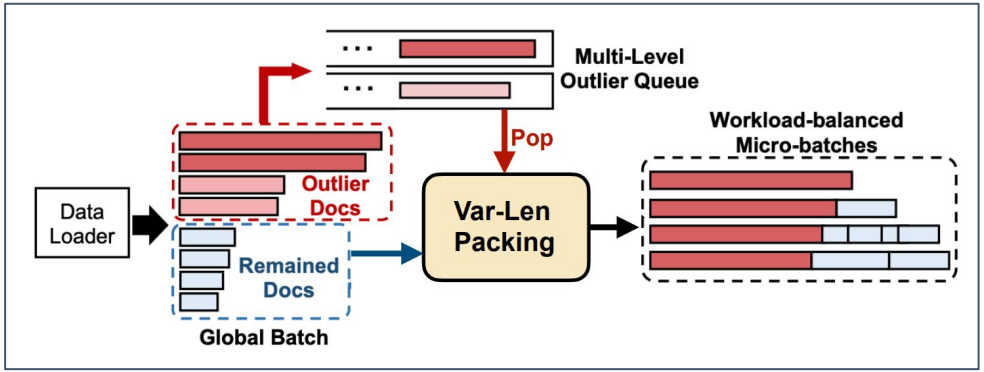
- Generalize results across frameworks
- More robust convergence impact testing

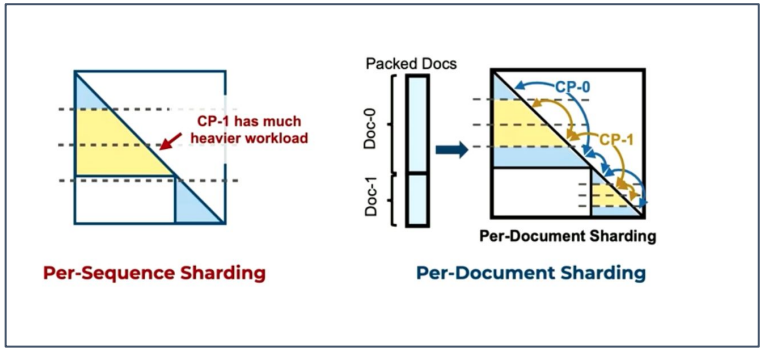
End



(c) V-Half


l_1
l_2
l_3
l_4
l_5
l_6
Parallel
V-Shape


Multi-Level
Outlier Queue
Pop
Workload-balanced
Micro-batches
Data
Loader
Outlier
Docs
Remained
Docs
Var-Len
Packing
Global Batch


CP-1 has much
heavier workload
Per-Sequence Sharding
Packed Docs
Doc-0
Doc-1
CP-0
CP-1
Per-Document Sharding
Per-Document Sharding